Week 4 Module

Date __/__/____

Topics to learn:-

(1) Introduction to Algorithm Analysis:

⇒ Algorithm analysis measures how efficient an algorithm is in terms of execution time and memory usage.

(2) Time Complexity:

Time Complexity tells how the execution time of an algorithm grows as the input size increases.

(3) Space Complexity:

Space Complexity measures the amount of extra memory required by an algorithm.

(4) Asymptotic Notations:

- Big O (O): Upper bound (Worst Case)
- Omega (Ω): lower bound (Best Case)
- Theta (Θ): Exact bound (Average/Exact case)

Date __/__/____

⑤ Best, Average and Worst Case :-

- Best Case :- Minimum execution time.
- Average Case :- Expected execution time.
- Worst Case :- Maximum execution time.

⑥ Rules to Calculate Complexity :-

- One loop running N times → $O(N)$
- Two nested loops → $O(N^2)$
- Logarithmic loop (dividing by 2 each time) → $O(\log N)$
- Sequential loops → Add complexities and keep the highest order.

⑦ Time Complexity of Loops :-

- for (i=0; i<n; i++) → $O(N)$
- for (i=1; i<=n; i*=2) → $O(\log N)$

⑧ Time Complexity of Nested Loops :-

```
for (i=0; i<n; i++)
  for (j=0; j<n; j++)
```

TC → $O(N^2)$

Date __/__/____

(9) Time Complexity of Logarithmic Loops :-

```
for (i=1; i<n; i*=2)
```

TC → O(log N)

(10) Space Complexity Basics :-

- No extra memory → O(1)
- Array of size N → O(N)
- Matrix of size N×N → O(N²)

Question 1):- Print numbers from 1 to N

```
#include <iostream>
using namespace std;

int main(){
   int N;
   cin >>N;

   for (int i = 1; i<=N; i++){
      cout <<i<< " ";
   }
   return 0;
}
```

Date __/__/____

Time Complexity = O(N)

Space Complexity = O(1)

Explanation :- The loop runs from 1 to N, printing each number exactly once. Since the number of iterations depends on N, the time complexity is O(N). Only a few variables are used, so the space complexity is O(1).

Optimized Approach :- There is no faster algorithm because every number from 1 to N must be printed.

```
PS C:\Users\pragy\OneDrive\Desktop\week module4> cd "c:\Users\pragy\OneDrive\Desktop\week mod
ule4\" ; if ($?) { g++ p1.cpp -o p1 } ; if ($?) { .\p1 }
5
1 2 3 4 5
PS C:\Users\pragy\OneDrive\Desktop\week module4>
```

be printed.

Question 2 :- Sum of first N Natural Numbers (Using loop) :-

```
#include <iostream>
using namespace std;

int main(){
    int N;
    cin >> N;

    long long sum = 0;

    for(int i = 1; i <= N; i++){
        sum += 1;
    }
```

Date ___/___/_____

```
    cout << sum;
    return 0;
}
```

Time Complexity = O(N)

Space Complexity = O(1)

Explanation:- The loop executes N times and adds each number to the sum. ∴, the running time increases linearly with N, giving O(N) time complexity. Only one variable is used to store the sum, so space complexity is O(1).

Optimized Approach 1:

Use the mathematical formula:

$$Sum = \frac{N(N+1)}{2}$$

This reduce the Time complexity to O(1).

```
> cd "c:\Users\pragy\OneDrive\Desktop\week module4\" ; if ($?) { g++ p2.cpp -o p2 } ; if ($?) { .\p2 }
10
55
PS C:\Users\pragy\OneDrive\Desktop\week module4>
```

Date __/__/____

Question 3:- Calculate A^B Using Loop

```
#include <iostream>
using namespace std;

int main(){
   int A, B;
   cin >>A>>B;

   long long result = 1;

   for (int i = 1; i<=B; i++) {
      result *= A;
   }
   cout << result;

   return 0;
}
```

Time Complexity = O(B)

Space Complexity = O(1)

Explanation:- The loop multiplies A by itself B times. Hence, the running time depends on the value of B, resulting in O(B) time complexity. Only one extra variable is used, so the space complexity is O(1).

Date __/__/____

Optimized Approach:-

A faster method is Binary Exponentiation (Fast Power), which computes the result in O(log B) time.

```
PS C:\Users\pragy\OneDrive\Desktop\week module4> cd "c:\Users\pragy\OneDrive\Desktop\week mod
ule4\" ; if ($?) { g++ p3.cpp -o p3 } ; if ($?) { .\p3 }
2 5
32
PS C:\Users\pragy\OneDrive\Desktop\week module4>
```